

Module 3: Semantic Analysis & Type Systems

SDD, SDTS, Synthesized/Inherited, Type Checking & Symbol Tables

Module III: Semantic Analysis & Intermediate Code – Complete 9-Topic Syllabus Tracker

- Topic 1:** Introduction to Semantic Analysis (Static vs Dynamic Checks)

Topic 3: Syntax-Directed Translation Schemes (SDTS Actions)

Topic 5: Three Address Code (Quadruples, Triples, Indirect Triples)

Topic 7: Type Checking for Expressions (Equivalence & Coercion)

Topic 9: Translation of Multi-Dimensional Array References
- Topic 2:** Syntax-Directed Definitions (SDD Semantic Rules)

Topic 4: SDTS for Declaration Processing & Symbol Tables

Topic 6: Types of Attributes (Synthesized vs. Inherited)

Topic 8: Intermediate Code Generation for Assignment Statements

Topic 1: Introduction to Semantic Analysis

Semantic Analysis verifies that the syntax tree conforms to the semantic rules of the language. While syntax ensures correct grammatical structure, semantics verifies logical validity:

Static Semantic Checks (Compile-Time): Type consistency, variable declared before use, scope resolution, identifier uniqueness.

Dynamic Semantic Checks (Run-Time): Division by zero, array out-of-bounds, null pointer dereferencing.

Topic 2 – 4: SDD, SDTS & Declaration Processing

Feature	Syntax-Directed Definition (SDD)	Syntax-Directed Translation Scheme (SDTS)
Representation	Context-Free Grammar + Semantic Rules associated with productions.	Context-Free Grammar with semantic action code embedded inside productions '{ ... }'.
Execution Timing	Specifies <i>what</i> value to compute (order determined by Dependency Graph topological sort).	Specifies <i>when</i> and in what exact sequence actions execute during parsing.

Step-by-Step SDTS: Processing Declarations `int a, b, c;`

```
D → T { L.in = T.type } L
T → int { T.type = integer }
T → float { T.type = real }
L → { L1.in = L.in } L1, id { enter(id.name, L.in) }
L → id { enter(id.name, L.in) }
```

Uses inherited attribute *L.in* to propagate base type down to all declared identifier names.

Topic 5: Three Address Code (TAC) Representations

TAC Form	Internal Data Fields	Key Advantages & Tradeoffs
1. Quadruples	<code>`(op, arg1, arg2, result)`</code> with explicit temporary names (<code>`t1`, `t2`</code>).	Easy to reorder and optimize; consumes extra memory for names.
2. Triples	<code>`(op, arg1, arg2)`</code> where intermediate results are referenced by index <code>`(0)`</code> .	Space-efficient; moving instructions requires updating all dependent indices.
3. Indirect Triples	Array of pointers to separate triple structures.	Optimizers reorder instructions by swapping array pointers without modifying triples.

Topic 6 & 7: Types of Attributes & Type Checking

1. Synthesized vs. Inherited Attributes:

Synthesized Attributes: Computed strictly from children nodes in parse tree ($A.s = f(Y_1.a, \dots, Y_k.a)$). Evaluated bottom-up in LR parsing.

Inherited Attributes: Computed from parent and/or left sibling nodes ($Y_j.i = f(A.a, Y_1.a, \dots, Y_{j-1}.a)$). Evaluated top-down.

2. Type Conversions & Equivalence:

Name Equivalence: Two types are equal if and only if they share the exact same named identifier.

Structural Equivalence: Two types are equal if they have identical internal field layouts and primitive types.

Widening (Coercion): ``int`` $\rightarrow$ ``float`` (implicit precision preservation).

Narrowing (Casting): ``float`` $\rightarrow$ ``int`` (explicit potential precision loss).

Topic 8 & 9: Intermediate Code Generation for Arrays & Assignments

1. 1D Array Reference Address Formula:

$$\text{Address}(A[i]) = \text{base} + (i - \text{low}) \times w$$

2. 2D Array Reference Address Formula (Row-Major Order):

$$\text{Address}(A[i_1, i_2]) = \text{base} + ((i_1 - \text{low}_1) \times n_2 + (i_2 - \text{low}_2)) \times w$$

Where $n_2 = \text{high}_2 - \text{low}_2 + 1$ is the number of columns, and w is element byte width.

3. General N -Dimensional Row-Major Linear Offset:

$$\text{Offset} = \left(\dots ((i_1 \cdot d_2 + i_2) \cdot d_3 + i_3) \dots \right) \cdot d_k + i_k$$

$$\text{Address} = \text{base} + \text{Offset} \times w$$



Q1. Write Quadruples and Triples for statement $a = b * -c + b * -c$. (8 Marks)

Quadruples:

- (0) $\text{uminus}, c, -, t1$
- (1) $*, b, t1, t2$
- (2) $\text{uminus}, c, -, t3$
- (3) $*, b, t3, t4$
- (4) $+, t2, t4, t5$
- (5) $=, t5, -, a$

Triples:

- (0) $\text{uminus}, c, -$
- (1) $*, b, (0)$
- (2) $\text{uminus}, c, -$
- (3) $*, b, (2)$
- (4) $+, (1), (3)$
- (5) $\text{assign}, a, (4)$